Lab 11

PL/pgSQL Named Blocks: Functions and Procedures

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

Contents

1 Lab Workflow **2**

2 Database Schema & Data Generation **5**

3 Introduction to Named Blocks **7**
3.1 User-Defined Functions 7
3.2 Stored Procedures 7

4 Parameter Modes **7**

5 Defining the Subprograms **8**

6 Execution Inside Anonymous Blocks **8**
6.1 Calling a Function 9
6.2 Calling a Procedure 9

7 Lab Tasks **10**

1 Lab Workflow

Repeat this subsection for every single lab.

1. Connect to the internet by running nmtui in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom.

3. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. In the Linux terminal:

```
psql
```

   Expected prompt:

```
cse=#
```

   2. Create Your Role and Database

   Copy and run the following commands inside PostgreSQL:

```
-- 1. Terminate existing database connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345'
  AND pid <> pg_backend_pid();

-- 2. Remove existing database
DROP DATABASE IF EXISTS cse4308_12345;

-- 3. Remove existing role
DROP ROLE IF EXISTS student_12345;

-- 4. Create role
CREATE ROLE student_12345 LOGIN;

-- 5. Set password
ALTER ROLE student_12345
WITH PASSWORD 'student12345';

-- 6. Create database
CREATE DATABASE cse4308_12345
OWNER student_12345;

-- 7. Grant privileges
GRANT ALL PRIVILEGES
ON DATABASE cse4308_12345
TO student_12345;
```

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

1. Connect to Your Database:

Use the following command to log in as the `student_12345` user:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Verify the Connection:

Verify that you are connected to the correct database with the following command:

```
\conninfo
```

Expected output:

```
Database: cse4308_12345, User: student_12345
```

3. Exit PostgreSQL:

```
\q
```

6. **Load the Lab SQL File (MUST be run outside psql)**

Load the SQL file that sets up the tables and inserts initial data.

1. Load the SQL File:

Run the following command to load the lab SQL file:

```
psql -h localhost -U student_12345 -d cse4308_12345 -f /home/cse/Downloads/
 lab_11_setup.sql
```

This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

7. **Reconnect and Verify Tables**

After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

1. Reconnect to Your Database:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Check Tables:

Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>;
```

8. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L11_T1.sql`. You will use this text editor to write your solutions, make sure to save it after writing each solution. The structure of this file will be:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

2 Database Schema & Data Generation

Instructions: Save this code as `lab_11_setup.sql` and run it using `\i lab_11_setup.sql` in your `psql` terminal. This script generates an e-commerce schema.

```sql
-- 1. CLEANUP
DROP TABLE IF EXISTS order_items CASCADE;
DROP TABLE IF EXISTS orders CASCADE;
DROP TABLE IF EXISTS restock_logs CASCADE;
DROP TABLE IF EXISTS products CASCADE;
DROP TABLE IF EXISTS customers CASCADE;

-- 2. CREATE TABLES
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    region VARCHAR(50),
    loyalty_tier VARCHAR(20) -- 'Bronze', 'Silver', 'Gold', 'Platinum'
);

CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    category VARCHAR(50),
    base_price NUMERIC(10, 2),
    stock_quantity INT
);

CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),
    order_date DATE,
    status VARCHAR(20), -- 'PENDING', 'PROCESSED', 'PARTIAL', 'CANCELLED'
    total_value NUMERIC(12, 2) DEFAULT 0.00
);

CREATE TABLE order_items (
    item_id SERIAL PRIMARY KEY,
    order_id INT REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id INT REFERENCES products(product_id),
    quantity INT,
    unit_price NUMERIC(10, 2)
);

CREATE TABLE restock_logs (
    log_id SERIAL PRIMARY KEY,
    product_id INT REFERENCES products(product_id),
    restock_date DATE,
    added_quantity INT
);

-- 3. DATA GENERATION
-- Insert 1000 Customers
INSERT INTO customers (name, region, loyalty_tier)
SELECT
    'Customer_' || id,
    (ARRAY['North', 'South', 'East', 'West'])[floor(random() * 4 + 1)],
    (ARRAY['Bronze', 'Silver', 'Gold', 'Platinum'])[floor(random() * 4 + 1)]
```

```sql
FROM generate_series(1, 1000) AS id;

-- Insert 200 Products
INSERT INTO products (name, category, base_price, stock_quantity)
SELECT
    'Product_' || id,
    (ARRAY['Electronics', 'Clothing', 'Groceries', 'Tools', 'Books'])[floor(random() *
     5 + 1)],
    ROUND((random() * 500 + 10)::numeric, 2),
    floor(random() * 100 + 10)::INT
FROM generate_series(1, 200) AS id;

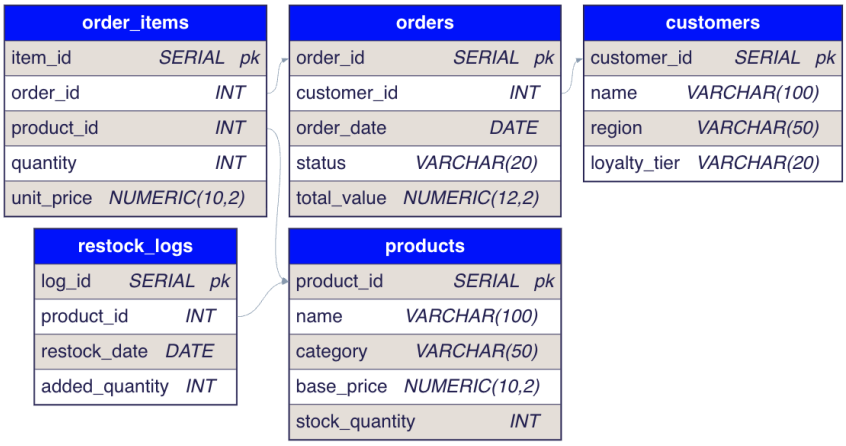
-- Insert 3000 Orders (Statuses randomized)
INSERT INTO orders (customer_id, order_date, status)
SELECT
    floor(random() * 1000 + 1)::INT,
    CURRENT_DATE - (floor(random() * 365)::INT),
    (ARRAY['PENDING', 'PROCESSED', 'PROCESSED', 'PARTIAL', 'CANCELLED'])[floor(random
    () * 5 + 1)]
FROM generate_series(1, 3000);

-- Insert 9000 Order Items (Averaging 3 items per order)
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT
    floor(random() * 3000 + 1)::INT,
    floor(random() * 200 + 1)::INT,
    floor(random() * 5 + 1)::INT,
    0.00
FROM generate_series(1, 9000) ON CONFLICT DO NOTHING;

-- Fix the unit_price in order_items based on products table
UPDATE order_items oi
SET unit_price = p.base_price
FROM products p
WHERE oi.product_id = p.product_id;

-- Initial aggregation to set order total_values
UPDATE orders o
SET total_value = sub.total
FROM (
    SELECT order_id, COALESCE(SUM(quantity * unit_price), 0) as total
    FROM order_items
    GROUP BY order_id
) sub
WHERE o.order_id = sub.order_id;
```

Listing 1: lab_11_setup.sql



3 Introduction to Named Blocks

Anonymous blocks (`DO $$`) evaluate procedural logic for single execution but are not persistently stored in the database schema. Persistent named blocks, referred to as subprograms, encapsulate procedural logic for reuse. In PostgreSQL, subprograms are classified into two primary constructs: **User-Defined Functions** and **Stored Procedures**.

3.1 User-Defined Functions

A named PL/pgSQL block that evaluates to a specific return type. Functions are invoked within expressions (e.g., a `SELECT` statement) and structurally require a `RETURN` or `RETURN QUERY` statement. They execute entirely within the transaction scope of the invoking query; they cannot execute Transaction Control Language (TCL) commands such as `COMMIT` or `ROLLBACK`.

3.2 Stored Procedures

A named PL/pgSQL block invoked via the `CALL` statement. Procedures do not evaluate to a direct return value and are capable of autonomous transaction control. This capability dictates their use for bulk data manipulation, schema migrations, and batch processing workflows.

4 Parameter Modes

Formal parameters in PL/pgSQL are assigned a mode that dictates the direction of data binding between the calling environment and the subprogram.

- `IN` **Mode (Implicit Default):** Specifies that the formal parameter accepts an actual argument from the caller. It is lexically immutable (read-only) within the subprogram body.
- `OUT` **Mode:** Specifies an uninitialized variable bound to the caller's environment. The subprogram assigns a value to it, which is transmitted back upon termination.
- `INOUT` **Mode:** A hybrid parameter that accepts an initialized actual argument from the caller, permits state mutation within the subprogram body, and transmits the modified state back to the caller upon termination.

5 Defining the Subprograms

Before invoking them dynamically, subprograms must be compiled into the schema.

```
CREATE OR REPLACE FUNCTION calculate_customer_lifetime_value(p_cust_id INT)
RETURNS NUMERIC AS $$
DECLARE
    v_total_spent NUMERIC;
BEGIN
    SELECT COALESCE(SUM(total_value), 0) INTO v_total_spent
    FROM orders
    WHERE customer_id = p_cust_id;

    RETURN v_total_spent;
END;
$$ LANGUAGE plpgsql;
```

Listing 2: Scalar Function Example

```
CREATE OR REPLACE PROCEDURE process_category_discount(
    p_category IN VARCHAR,      -- Read-only constraint
    p_discount_budget INOUT NUMERIC,   -- Mutated and returned to caller
    p_budget_status OUT VARCHAR -- Assigned and returned to caller
)
LANGUAGE plpgsql AS $$
DECLARE
    v_product_count INT;
BEGIN
    -- Utilize IN parameter for scalar subquery
    SELECT COUNT(*) INTO v_product_count
    FROM products
    WHERE category = p_category;

    -- Transform the INOUT parameter
    p_discount_budget := p_discount_budget - (v_product_count * 50.00);

    -- Initialize the OUT parameter
    IF p_discount_budget > 0 THEN
        p_budget_status := 'BUDGET_SURPLUS';
    ELSE
        p_budget_status := 'BUDGET_DEFICIT';
    END IF;

    -- Procedural Transaction Control
    COMMIT;
END;
$$;
```

Listing 3: Procedure Example with Mixed Modes and TCL

6 Execution Inside Anonymous Blocks

This is the standard methodology for testing or dynamically invoking procedural logic without saving a new script to the database schema. Because anonymous blocks cannot return result sets directly to the client interface like a standard `SELECT` statement, you

must utilize the `DECLARE` section to capture outputs and `RAISE NOTICE` to observe the results.

6.1 Calling a Function

Functions evaluate to a value. Inside a block, you must either assign this evaluated value to a declared variable using the `:=` operator, or use `SELECT ... INTO`.

```
DO $$
DECLARE
    -- 1. Declare a variable to hold the function's return payload
    v_ltv NUMERIC;
BEGIN
    -- 2. Assign the function evaluation directly to the variable
    v_ltv := calculate_customer_lifetime_value(1);

    -- 3. Output the state
    RAISE NOTICE 'Customer LTV: %', v_ltv;
END;
$$;
```

Listing 4: Calling a Function

6.2 Calling a Procedure

Procedures are invoked using the `CALL` statement. When a procedure utilizes `OUT` or `INOUT` parameters, you cannot pass raw literals into those parameter slots. You must pass **variables by reference** so the procedure has a memory address to write the modified state back to.

```
DO $$
DECLARE
    -- 1. Initialize variables for INOUT parameters
    v_promo_budget NUMERIC := 10000.00;

    -- 2. Declare uninitialized variables for OUT parameters
    v_status VARCHAR;
BEGIN
    -- 3. Execute the procedure, passing the variables by reference
    CALL process_category_discount(
        'Electronics',   -- IN argument (literal is fine here)
        v_promo_budget,  -- INOUT argument (must be a variable)
        v_status         -- OUT argument (must be a variable)
    );

    -- 4. The variables in this block's scope now hold the mutated states
    RAISE NOTICE 'Remaining Budget: %, Status: %', v_promo_budget, v_status;
END;
$$;
```

Listing 5: Calling a Procedure

7 Lab Tasks

1. Write a **Stored Procedure** named `evaluate_loyalty_tiers` that accepts an `INOUT` parameter `p_upgraded_count` (initialized to 0). The procedure must loop through all customers, calculate their lifetime spending (sum of `total_value` for `'PROCESSED'` orders), and upgrade their `loyalty_tier` based on the following thresholds: $> \$10,000$ to `'Platinum'`, $> \$5,000$ to `'Gold'`, $> \$1,000$ to `'Silver'`. Do not downgrade anyone. If a customer's tier is changed, increment the `INOUT` counter.

   Solution:

```
CREATE OR REPLACE PROCEDURE evaluate_loyalty_tiers(INOUT p_upgraded_count INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_cust RECORD;
BEGIN
    FOR v_cust IN
        SELECT c.customer_id, c.loyalty_tier, COALESCE(SUM(o.total_value), 0) AS
     spend
        FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id AND
     o.status = 'PROCESSED'
        GROUP BY c.customer_id, c.loyalty_tier
    LOOP
        IF v_cust.spend > 10000 AND v_cust.loyalty_tier != 'Platinum' THEN
            UPDATE customers SET loyalty_tier = 'Platinum' WHERE customer_id =
    v_cust.customer_id;
            p_upgraded_count := p_upgraded_count + 1;
        ELSIF v_cust.spend > 5000 AND v_cust.loyalty_tier IN ('Bronze', 'Silver'
    ) THEN
            UPDATE customers SET loyalty_tier = 'Gold' WHERE customer_id =
    v_cust.customer_id;
            p_upgraded_count := p_upgraded_count + 1;
        ELSIF v_cust.spend > 1000 AND v_cust.loyalty_tier = 'Bronze' THEN
            UPDATE customers SET loyalty_tier = 'Silver' WHERE customer_id =
    v_cust.customer_id;
            p_upgraded_count := p_upgraded_count + 1;
        END IF;
    END LOOP;
    COMMIT;
END;
$$;
```

2. Create a **Stored Procedure** named `consolidate_customer_orders` that takes an `IN` parameter `p_customer_id` and an `OUT` parameter `p_master_order_id`. Find all `'PENDING'` orders for this customer. If they have fewer than 2 pending orders, set the OUT parameter to `-1` and exit. Otherwise, designate the oldest pending order as the "master" order. Transfer all other pending `order_items` to the master order, delete the emptied orders, and update the master order's `total_value`.

   Solution:

```
CREATE OR REPLACE PROCEDURE consolidate_customer_orders(IN p_customer_id INT,
    OUT p_master_order_id INT)
LANGUAGE plpgsql AS $$
DECLARE
```

```
    v_ord RECORD;
BEGIN
    SELECT order_id INTO p_master_order_id FROM orders
    WHERE customer_id = p_customer_id AND status = 'PENDING' ORDER BY order_date
     ASC LIMIT 1;

    IF NOT FOUND OR (SELECT COUNT(*) FROM orders WHERE customer_id =
    p_customer_id AND status = 'PENDING') < 2 THEN
        p_master_order_id := -1; RETURN;
    END IF;

    FOR v_ord IN SELECT order_id FROM orders WHERE customer_id = p_customer_id
    AND status = 'PENDING' AND order_id != p_master_order_id LOOP
        UPDATE order_items SET order_id = p_master_order_id WHERE order_id =
    v_ord.order_id;
        DELETE FROM orders WHERE order_id = v_ord.order_id;
    END LOOP;

    UPDATE orders SET total_value = (SELECT COALESCE(SUM(quantity * unit_price),
     0) FROM order_items WHERE order_id = p_master_order_id)
    WHERE order_id = p_master_order_id;
    COMMIT;
END;
$$;
```

3. Write a **Stored Procedure** named `dynamic_pricing_adjustment` with two `OUT` parameters: `p_increased_count` and `p_decreased_count`. Iterate through all products. If the total quantity ever sold is greater than 100, increase the `base_price` by 10%. If the total sold is strictly less than 10, decrease the `base_price` by 15%. Track the number of products modified.

   Solution:

```
CREATE OR REPLACE PROCEDURE dynamic_pricing_adjustment(OUT p_increased_count INT
    , OUT p_decreased_count INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_prod RECORD;
BEGIN
    p_increased_count := 0; p_decreased_count := 0;

    FOR v_prod IN SELECT p.product_id, COALESCE(SUM(oi.quantity), 0) AS qty
                  FROM products p LEFT JOIN order_items oi USING (product_id)
    GROUP BY p.product_id LOOP
        IF v_prod.qty > 100 THEN
            UPDATE products SET base_price = base_price * 1.10 WHERE product_id
    = v_prod.product_id;
            p_increased_count := p_increased_count + 1;
        ELSIF v_prod.qty < 10 THEN
            UPDATE products SET base_price = base_price * 0.85 WHERE product_id
    = v_prod.product_id;
            p_decreased_count := p_decreased_count + 1;
        END IF;
    END LOOP;
    COMMIT;
END;
```

```
$$;
```

4. Design a **Stored Procedure** `cancel_order_and_restore(IN p_order_id, OUT p_refund_amount)`. Verify the order is not already `'CANCELLED'`. Loop through every item in that order. Add the purchased quantity back to `products.stock_quantity`, insert a record into `restock_logs`, and accumulate the financial value for the refund. Finally, update the order status to `'CANCELLED'`.

   Solution:

```
CREATE OR REPLACE PROCEDURE cancel_order_and_restore(IN p_order_id INT, OUT
    p_refund_amount NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
    v_item RECORD;
BEGIN
    p_refund_amount := 0.00;

    IF (SELECT status FROM orders WHERE order_id = p_order_id) = 'CANCELLED'
    THEN
        RAISE EXCEPTION 'Order % is already cancelled.', p_order_id;
    END IF;

    FOR v_item IN SELECT product_id, quantity, (quantity * unit_price) AS cost
    FROM order_items WHERE order_id = p_order_id LOOP
        UPDATE products SET stock_quantity = stock_quantity + v_item.quantity
    WHERE product_id = v_item.product_id;
        INSERT INTO restock_logs (product_id, restock_date, added_quantity)
    VALUES (v_item.product_id, CURRENT_DATE, v_item.quantity);
        p_refund_amount := p_refund_amount + v_item.cost;
    END LOOP;

    UPDATE orders SET status = 'CANCELLED' WHERE order_id = p_order_id;
    COMMIT;
END;
$$;
```

5. Write a **Table-Valued Function** `get_cross_sell_recommendations(p_target_product_id)`. Identify all orders containing this target product. Find all *other* products purchased within those exact same orders. Return a table of `(rec_product_id, rec_name, co_buy_count)` for the top 3 most frequently co-purchased items, sorted descending by count.

   Solution:

```
CREATE OR REPLACE FUNCTION get_cross_sell_recommendations(p_target_product_id
    INT)
RETURNS TABLE (rec_product_id INT, rec_name VARCHAR, co_buy_count INT)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT oi.product_id, p.name, COUNT(*)::INT as frequency
    FROM order_items oi JOIN products p USING (product_id)
    WHERE oi.order_id IN (SELECT order_id FROM order_items WHERE product_id =
    p_target_product_id)
```

```
      AND oi.product_id != p_target_product_id
    GROUP BY oi.product_id, p.name ORDER BY frequency DESC LIMIT 3;
END;
$$;
```

6. Implement a **Table-Valued Function** `category_performance_report` with no parameters. Compute the total gross revenue (from `'PROCESSED'` orders) and identify the single best-selling product (by quantity) for every category. Return `(category_name, total_gross_revenue, best_selling_product_name)`.

   Solution:

```
CREATE OR REPLACE FUNCTION category_performance_report()
RETURNS TABLE (category_name VARCHAR, total_gross_revenue NUMERIC,
    best_selling_product_name VARCHAR)
LANGUAGE plpgsql AS $$
BEGIN
    -- Optimized: Return query directly using CTEs, completely avoiding
    procedural loops
    RETURN QUERY
    WITH CatRev AS (
        SELECT p.category, COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS rev
        FROM products p
        LEFT JOIN order_items oi USING (product_id)
        LEFT JOIN orders o ON oi.order_id = o.order_id AND o.status = 'PROCESSED
    '
        GROUP BY p.category
    ),
    BestSellers AS (
        SELECT DISTINCT ON (p.category) p.category, p.name
        FROM products p JOIN order_items oi USING (product_id) JOIN orders o
    USING (order_id)
        WHERE o.status = 'PROCESSED'
        GROUP BY p.category, p.name ORDER BY p.category, SUM(oi.quantity) DESC
    )
    SELECT cr.category, cr.rev, COALESCE(bs.name, 'N/A'::VARCHAR)
    FROM CatRev cr LEFT JOIN BestSellers bs USING (category);
END;
$$;
```

7. Write a **Stored Procedure** `auto_restock_depleted(IN p_threshold, IN p_restock_amount OUT p_total_cost)`. Loop through products where `stock_quantity` < `p_threshold`. Increase the stock by `p_restock_amount`, insert a log into `restock_logs`, and accumulate the total restocking cost (wholesale cost is 40% of `base_price`).

   Solution:

```
CREATE OR REPLACE PROCEDURE auto_restock_depleted(IN p_threshold INT, IN
    p_restock_amount INT, OUT p_total_cost NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
    v_prod RECORD;
BEGIN
    p_total_cost := 0.00;
```

```
    FOR v_prod IN SELECT product_id, base_price FROM products WHERE
    stock_quantity < p_threshold LOOP
        UPDATE products SET stock_quantity = stock_quantity + p_restock_amount
    WHERE product_id = v_prod.product_id;
        INSERT INTO restock_logs (product_id, restock_date, added_quantity)
    VALUES (v_prod.product_id, CURRENT_DATE, p_restock_amount);
        p_total_cost := p_total_cost + ((v_prod.base_price * 0.40) *
    p_restock_amount);
    END LOOP;
    COMMIT;
END;
$$;
```

8. Develop a **Stored Procedure** `reward_vip_customers(IN p_top_n)`. Identify the top N customers by total lifetime spending (`'PROCESSED'` orders). For each VIP, find their most expensive purchased product and generate a complimentary order containing 1 unit of that product at a `unit_price` of 0. Deduct the stock.

   Solution:

```
CREATE OR REPLACE PROCEDURE reward_vip_customers(IN p_top_n INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_cust RECORD; v_new_order_id INT;
BEGIN
    FOR v_cust IN
        SELECT c.customer_id,
               (SELECT oi.product_id FROM order_items oi JOIN orders o USING (
    order_id)
                WHERE o.customer_id = c.customer_id ORDER BY oi.unit_price DESC
    LIMIT 1) AS fav_prod
        FROM orders c WHERE status = 'PROCESSED' GROUP BY c.customer_id ORDER BY
     SUM(total_value) DESC LIMIT p_top_n
    LOOP
        IF v_cust.fav_prod IS NOT NULL THEN
            INSERT INTO orders (customer_id, order_date, status, total_value)
            VALUES (v_cust.customer_id, CURRENT_DATE, 'PROCESSED', 0.00)
    RETURNING order_id INTO v_new_order_id;

            INSERT INTO order_items (order_id, product_id, quantity, unit_price)
            VALUES (v_new_order_id, v_cust.fav_prod, 1, 0.00);

            UPDATE products SET stock_quantity = stock_quantity - 1 WHERE
    product_id = v_cust.fav_prod;
        END IF;
    END LOOP;
    COMMIT;
END;
$$;
```

9. Write a **Table-Valued Function** `detect_suspicious_activity`. Return `(suspicious_custome incident_date, daily_order_count, daily_total)`. Find customers who have placed more than 3 orders on any single calendar date where the combined total value of those orders exceeds $1,000.

Solution:

```
CREATE OR REPLACE FUNCTION detect_suspicious_activity()
RETURNS TABLE (suspicious_customer_id INT, incident_date DATE, daily_order_count
     INT, daily_total NUMERIC)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT customer_id, order_date, COUNT(*)::INT, SUM(total_value)
    FROM orders GROUP BY customer_id, order_date
    HAVING COUNT(*) > 3 AND SUM(total_value) > 1000.00;
END;
$$;
```

10. Create a **Scalar Function** `project_future_revenue(IN p_months_ahead)`. Calculate average daily revenue over the last 30 days and compare it to the prior 30 days to get a growth rate. Using the compound interest formula, project the total revenue for the 30-day period occurring exactly `p_months_ahead` in the future.

Solution:

```
CREATE OR REPLACE FUNCTION project_future_revenue(p_months_ahead INT) RETURNS
    NUMERIC
LANGUAGE plpgsql AS $$
DECLARE
    v_recent NUMERIC; v_past NUMERIC; v_rate NUMERIC;
BEGIN
    SELECT COALESCE(SUM(total_value), 0) INTO v_recent FROM orders WHERE
    order_date >= CURRENT_DATE - 30;
    SELECT COALESCE(SUM(total_value), 0) INTO v_past FROM orders WHERE
    order_date >= CURRENT_DATE - 60 AND order_date < CURRENT_DATE - 30;

    v_rate := CASE WHEN v_past = 0 THEN 0 ELSE (v_recent - v_past) / v_past END;
    RETURN ROUND(v_recent * POWER((1.0 + v_rate), p_months_ahead), 2);
END;
$$;
```

11. Implement a **Stored Procedure** `apply_bulk_discounts(IN p_min_qty, IN p_discount_pct)`. Iterate through all `'PENDING'` orders. If an item's requested `quantity` $\geq$ `p_min_qty`, calculate the discount and update its `unit_price`. Recalculate and update the order's `total_value` if modified.

Solution:

```
CREATE OR REPLACE PROCEDURE apply_bulk_discounts(IN p_min_qty INT, IN
    p_discount_pct NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
    v_ord RECORD; v_modified BOOLEAN;
BEGIN
    FOR v_ord IN SELECT order_id FROM orders WHERE status = 'PENDING' LOOP
        v_modified := FALSE;

        -- Update items and track if changes were made
        UPDATE order_items SET unit_price = unit_price * (1.0 - (p_discount_pct
    / 100.0))
```

```
        WHERE order_id = v_ord.order_id AND quantity >= p_min_qty RETURNING
    order_id INTO v_modified;

        -- Only recalculate totals for orders that were actually modified
        IF v_modified THEN
            UPDATE orders SET total_value = (
                SELECT COALESCE(SUM(quantity * unit_price), 0) FROM order_items
    WHERE order_id = v_ord.order_id
            ) WHERE order_id = v_ord.order_id;
        END IF;
    END LOOP;
    COMMIT;
END;
$$;
```

12. Design a **Stored Procedure** `anonymize_inactive_customers(OUT p_anonymized_count)`. If a customer has never placed an order or their last order was over 730 days ago, execute an anonymization process: change their `name` to `'ANONYMIZED'`, `region` to `'UNKNOWN'`, and `loyalty_tier` to `'Bronze'`. Count updated rows.

Solution:

```
CREATE OR REPLACE PROCEDURE anonymize_inactive_customers(OUT p_anonymized_count
    INT)
LANGUAGE plpgsql AS $$
DECLARE
    v_cust RECORD;
BEGIN
    p_anonymized_count := 0;

    FOR v_cust IN
        SELECT c.customer_id, COALESCE(CURRENT_DATE - MAX(o.order_date), 9999)
    AS days_inactive
        FROM customers c LEFT JOIN orders o USING (customer_id) GROUP BY c.
    customer_id
    LOOP
        IF v_cust.days_inactive > 730 THEN
            UPDATE customers SET name = 'ANONYMIZED', region = 'UNKNOWN',
    loyalty_tier = 'Bronze'
            WHERE customer_id = v_cust.customer_id;
            p_anonymized_count := p_anonymized_count + 1;
        END IF;
    END LOOP;
    COMMIT;
END;
$$;
```